

Lecture 5: Data Visualization

PSTAT100: Data Science - Concepts and Analysis

John Inston 

johninston@ucsb.edu

University of California, Santa Barbara

April 15, 2026

Overview

Aims of the lecture

- Introduce the concept of **data visualization** and its importance in data analysis.
- Provide an overview of **basic visualizations** and when to use them.
- Introduce the `matplotlib` and `seaborn` libraries for creating visualizations in Python.
- Explore various **style** and **formatting** features including:
 - **Themes.**
 - **Variable Formatting**
 - **Facets.**

Required Libraries

In this lecture we will be using the following **libraries**:

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
```

Recap: Data Preparation

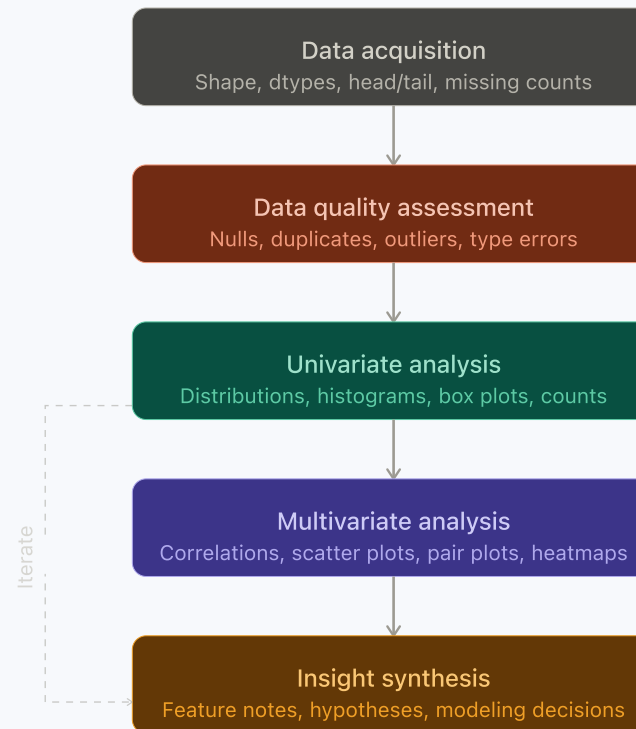
Data Preparation

- So far we have covered:
 - **Data acquisition** and understanding.
 - **Data preparation.**
 - **Tidy data.**
 - Quality assessment.
 - **Data cleaning.**

Next Step: Exploration

- Perform **exploratory data analysis (EDA)**.
- Learn data **properties** and **relationships**.

EDA Workflow.



Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA)

What is EDA?

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the process of analyzing and **visualizing** data to understand its structure, identify patterns, and uncover insights.

EDA Steps (Topic of next lecture)

1. Perform **univariate analysis** to understand the distribution of individual variables.
2. Perform **multivariate analysis** to explore relationships between variables.
3. Develop insights to inform modeling and deeper analysis.

Data Visualization

- A key component of EDA is **data visualization**.
- Python's standard library does not include a full visualization system, so we use dedicated libraries such as:
 - `matplotlib`, `seaborn`, `altair`, and `plotly`.

Data Visualization

Data Visualization

Why Visualize Data?

- Improve **interpretability** of data.
- Identify **patterns** and **relationships**.
- **Communicate** findings effectively.
- Explore data in an **interactive** way.

Principles of Effective Data Visualization

- **Novel**
- **Informative**
- **Efficient**
- **Aesthetic**

Always ask yourself, what is this visualization for? What is it contributing?

Types of Visualization

Exploratory vs Presentation

- **Exploratory visualizations**
 - Used during the data analysis process to explore and understand the data.
 - Often created quickly and may not be polished or refined.
- **Presentation visualizations**
 - Used to communicate findings to an audience.
 - Typically more polished and refined, with a focus on clarity and aesthetics.

Static vs Interactive vs Animated

- **Static visualizations:** These are fixed images that do not allow for user interaction.
- **Interactive visualizations:** These allow users to interact with the data, such as zooming, filtering, and hovering for more information.
- **Animated visualizations:** These show changes in data over time or through different conditions.

Visualizations in Python

matplotlib

- The foundational visualization library in Python.
- Almost every other Python visualization library — including `seaborn` — is built on top of it.
- It is an **imperative** visualization library, meaning:
 - Total control over all plot details.
 - Often more verbose for common plots.
- Strong for fine-grained customization and publication-style figures.

seaborn

- **Declarative** statistical visualization library for Python.
 - Similar to `ggplot2` in R.

- It is a **high-level statistical plotting API** on top of `matplotlib`, meaning:
 - You can create common statistical visualizations with much less code.
 - It provides useful defaults for **themes**, **color palettes**, and **grouped comparisons**.
- Well suited for EDA and rapid iteration.

altair

- A **declarative** statistical visualization library for Python, based on Vega and Vega-Lite.
- It allows you to create **interactive visualizations** with a simple and intuitive syntax.
- Good for EDA, interactive documents, and reproducible chart specifications.

Plan for upcoming lectures

Lecture 5 (Today)

- Summary of **standard visualizations** and when to use them.
 - Univariate, bivariate, multivariate.
 - Categorical vs continuous variables.
- Outline standard basic (static) visualization production.
 - Summarize foundations in `matplotlib`.
 - Focus on `seaborn` for EDA visualizations.

Lecture 6 (Thursday)

- **Exploratory Data Analysis (EDA).**
- Descriptive statistics.
- Complex `seaborn` visualizations.
- Interactive visualizations with `altair`.

Basic Visualizations

Standard Visualizations

- We use different plots to visualize different types of data and relationships.
 - Univariate and continuous variables:
 - **Box plots, Violin plots:**
 - Visualize the descriptive statistics of a continuous variable.
 - **Histograms, Density plots**
 - Visualize the distribution of a continuous variable.
 - Univariate and categorical variables:
 - **Bar plots, Count plots**
 - Visualize the count or proportion of categories in a categorical variable.
 - Bivariate and continuous:
 - **Scatter plots**
 - Visualize the relationship between two continuous variables.
 - **Line plots**
 - Visualize the relationship between two continuous variables over time or another continuous dimension.

Example Data: `penguins`

- We will be using the `penguins` dataset loaded from the `seaborn` library.
- Contains measurements of penguins from three different species: Adelie, Chinstrap, and Gentoo.

```
1 penguins = sns.load_dataset("penguins")
2 print(penguins.shape)
3 penguins.head()
```

(344, 7)

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

Object-Oriented Interface

Pyplot vs Object-Oriented Interface

- **Pyplot interface:** A state-based interface that allows you to create plots using a simple, MATLAB-like syntax. It is suitable for quick and simple visualizations.
- **Object-oriented interface:** A more flexible and powerful interface that allows you to create complex and customized plots by working with figure and axes objects directly. It is recommended for more complex visualizations and when you need more control over the plot.
 - We will primarily be using the object-oriented interface in this course.

Only covering the basics of `matplotlib`

- We will focus on plotting in `seaborn` and `altair` in this course.
- We only cover what we need to know in `matplotlib`.

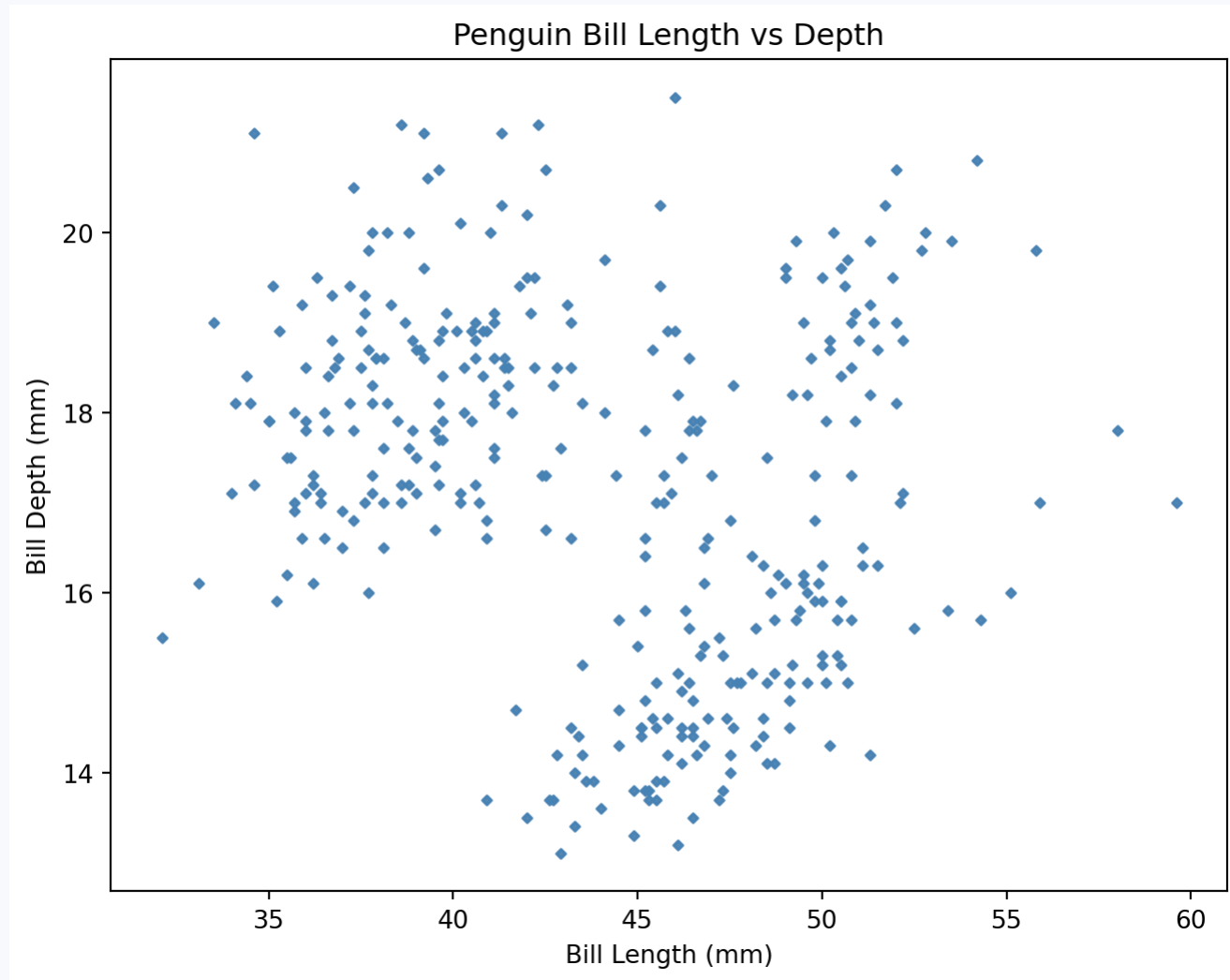
matplotlib Scatter Plots

Scatter Plots

- A **scatter plot** pairs two continuous variables to visualize their relationship.
- We produce scatter plots in `matplotlib` as follows:
 - Create a figure and axis object using `plt.subplots()`.
 - Call the appropriate plotting function on the axis object (e.g. `ax.scatter()`).
 - Customize the plot using axis methods `ax.set()`.
 - Display the plot using `plt.show()`.

```
1 fig, ax = plt.subplots(figsize=(8, 6))          # Create figure and axis objects
2 ax.scatter (                                    # Create scatter plot
3     "bill_length_mm", "bill_depth_mm",
4     data=penguins,
5     color="steelblue",                          # Set point color
6     alpha=0.95,                                  # Set point transparency
7     s=7,                                          # Set point size
8     marker="D"                                    # Set point type
9 )
10 ax.set(                                          # Set labels and title
11     xlabel="Bill Length (mm)",
12     ylabel="Bill Depth (mm)",
13     title="Penguin Bill Length vs Depth"
14 )
15 plt.show()                                     # Show the plot
```

matplotlib Scatter Plots



Scatter plot of bill length vs bill depth for the penguins dataset. (`matplotlib`)

seaborn Scatter Plots

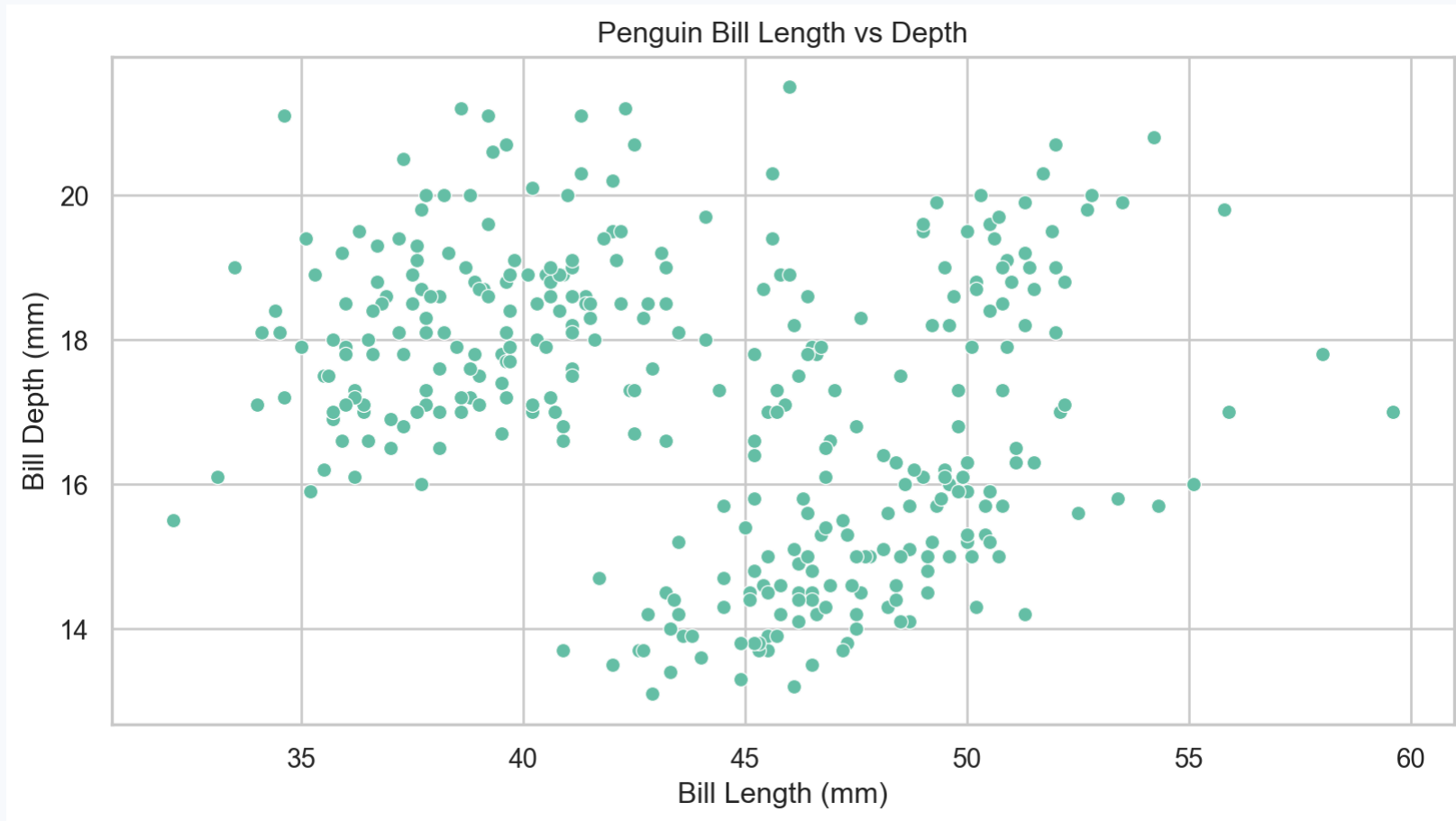
What makes **seaborn** so great?

- Uses the same underlying plotting functions as **matplotlib** but provides a **higher-level interface** for creating more **complex** and **informative** visualizations with less code.
- Provides built-in themes and color palettes to make it easy to create **attractive** visualizations.
- Provides functions for **visualizing statistical relationships**, such as regression lines and confidence intervals.

Scatter Plots in **seaborn**

```
1 sns.set_theme(style="whitegrid", palette="Set2")           # Set theme
2 ax = sns.scatterplot(                                     # Create scatter plot
3     x="bill_length_mm", y="bill_depth_mm",
4     data=penguins
5 )
6 ax.set(
7     xlabel="Bill Length (mm)",
8     ylabel="Bill Depth (mm)",
9     title="Penguin Bill Length vs Depth")
10 plt.show()                                              # Show the plot
```

seaborn Scatter Plots



Scatter plot of bill length vs bill depth for the penguins dataset. ([seaborn](#))

Variable Based Formatting

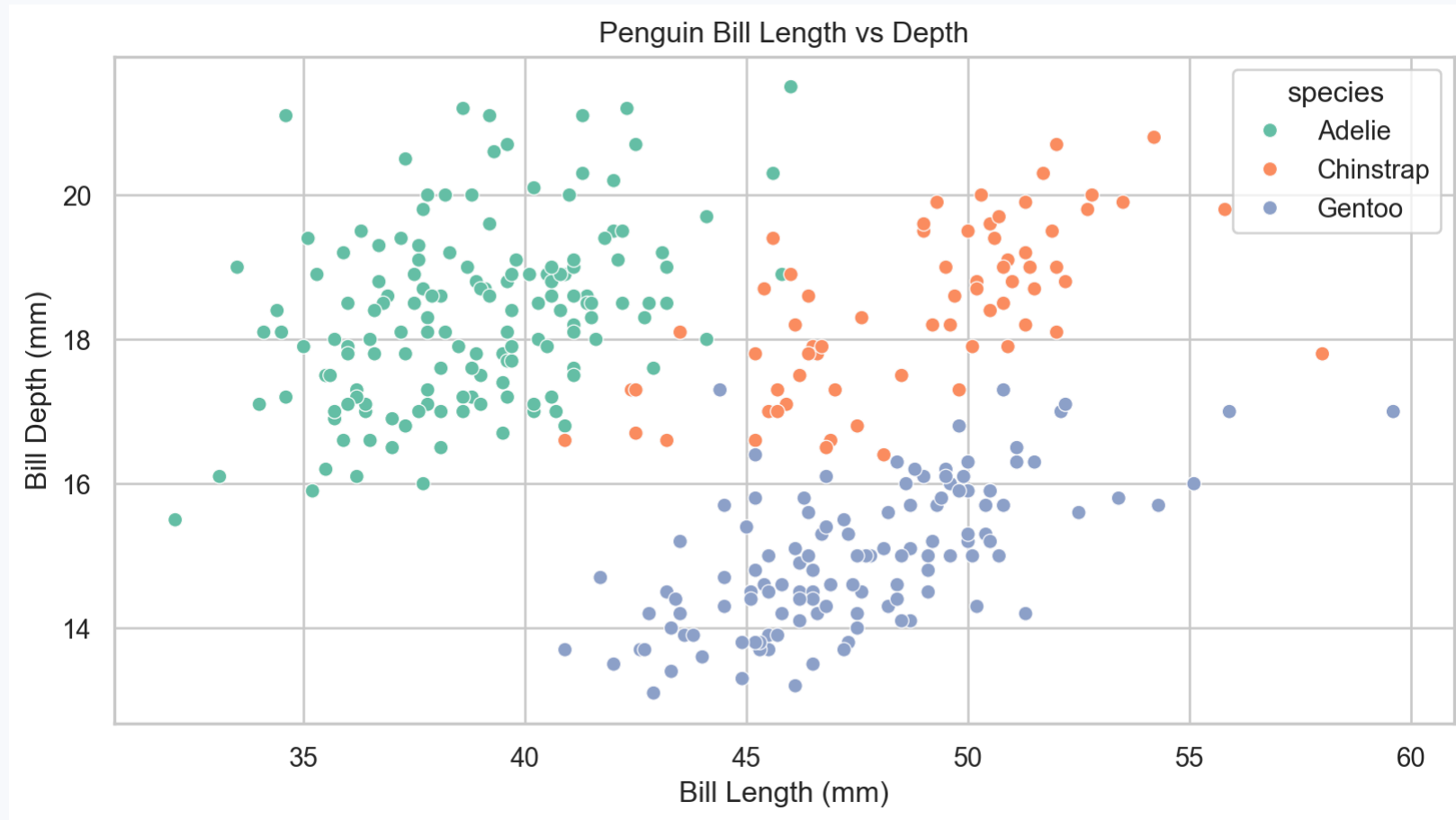
Variable Based Formatting

- Often we would like to examine the influence of a **third categorical variable** on the relationship between two continuous variables.
- We can use **variable-based formatting** to color points by species in the penguins dataset.

Scatter Plots with Variable Based Formatting in **seaborn**

```
1 ax = sns.scatterplot(  
2     x="bill_length_mm", y="bill_depth_mm",  
3     data=penguins,  
4     hue="species"                # Color points by species  
5 )  
6 ax.set(  
7     xlabel="Bill Length (mm)",  
8     ylabel="Bill Depth (mm)",  
9     title="Penguin Bill Length vs Depth")  
10 plt.show()
```

Variable Based Formatting



Scatter plot of bill length vs bill depth grouped by species for the penguins dataset. (`seaborn`)

Adding Best Fit Lines

What are best fit lines?

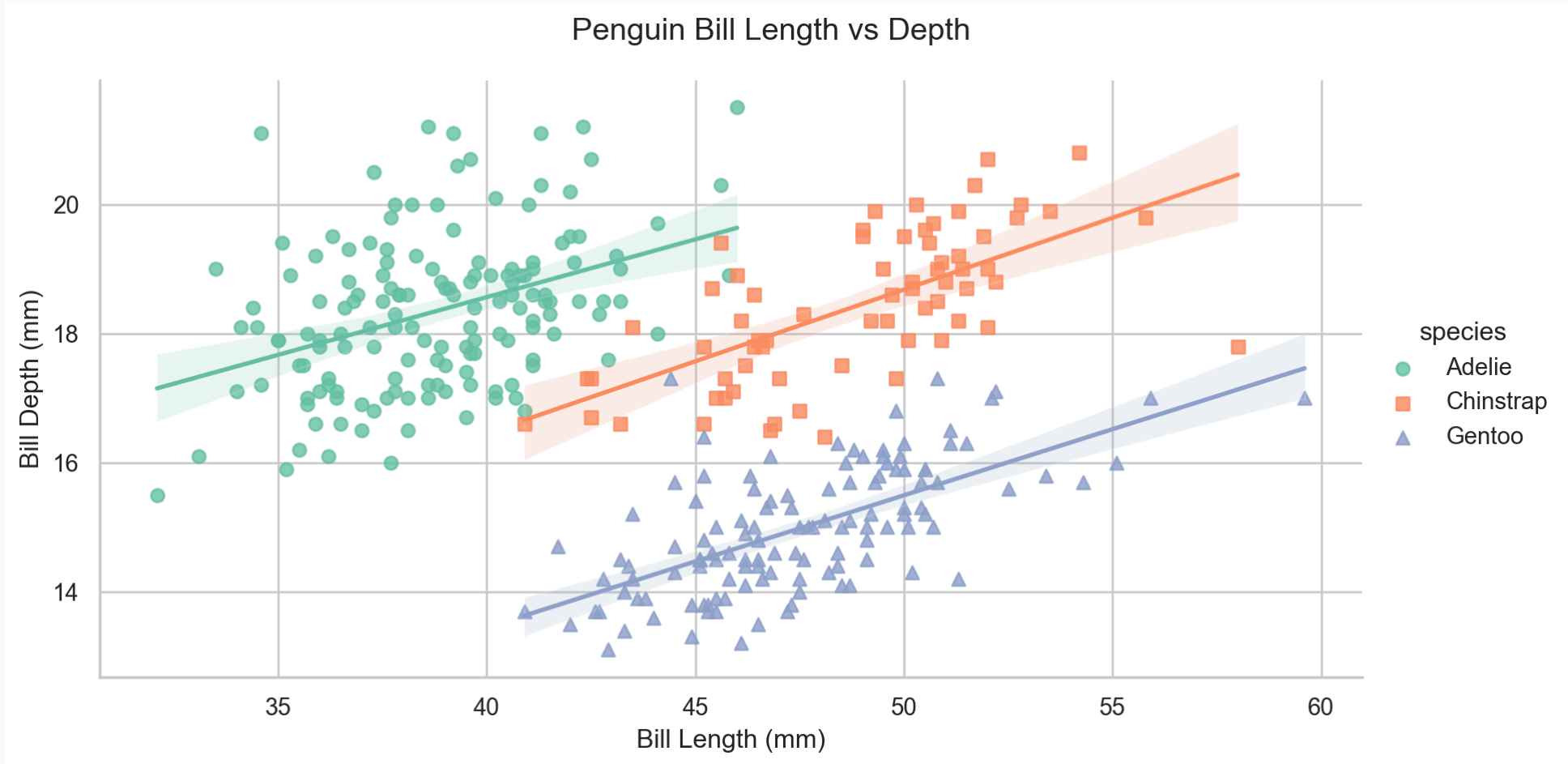
- A **best fit line** is a line that best fits the data points in a scatter plot, showing the overall trend of the relationship between the two variables.
- In `seaborn`, we can produce a scatter plot with a best fit line using the `lmplot()` function.

For now, treat best fit lines as descriptive trend summaries; we will cover regression assumptions, estimation, and interpretation in detail in future weeks.

`seaborn` Regression Lines

```
1 g = sns.lmplot(                                # Scatter with regression line
2     x="bill_length_mm", y="bill_depth_mm",
3     data=penguins,
4     hue="species",
5     markers=["o", "s", "^"],                    # Set point types
6     line_kws={"linewidth": 2},                  # Set line width
7     height=5, aspect=1.8                        # Change figure size
8 )
9 g.set_axis_labels("Bill Length (mm)", "Bill Depth (mm)")
10 g.fig.subplots_adjust(top=0.9)
11 g.fig.suptitle("Penguin Bill Length vs Depth")
12 plt.show()
```

Adding Best Fit Lines



Scatter plot of bill length vs bill depth grouped by species with best fit lines and confidence intervals for the penguins dataset. ([seaborn](#))

Bar Plots

What are bar plots?

- A **bar plot** is a chart that presents categorical data with rectangular bars, where the length of each bar is proportional to the value it represents.
- They are often used to represent the count or proportion of categories in a categorical variable.
- Bar plots in `matplotlib` are driven by the summary data frame we provide. In this case, we compute species counts in advance.

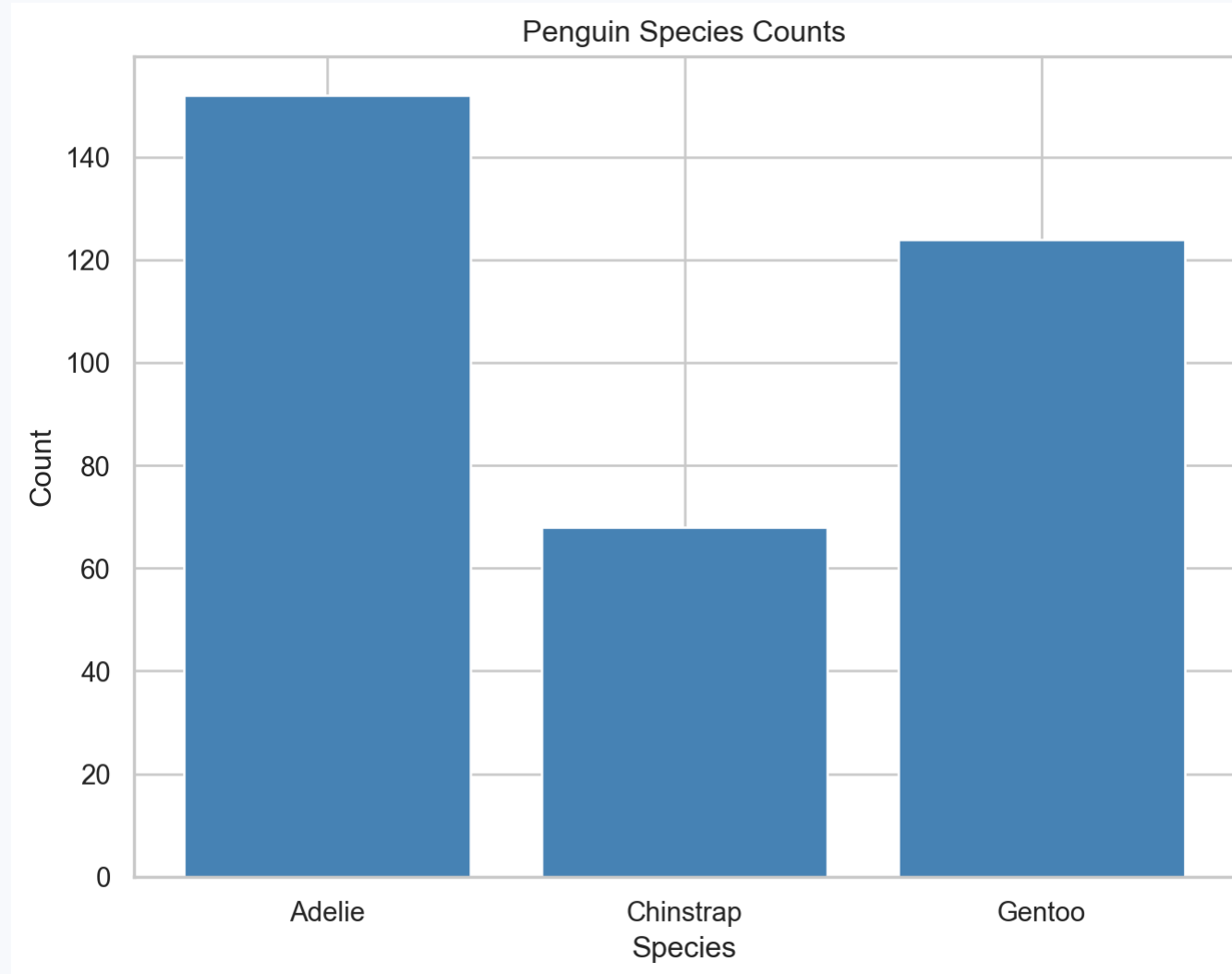
`matplotlib` Bar Plots

```
1 bp1_data = penguins.groupby("species").size(  
2     ).reset_index(name="count")  
3 print(bp1_data)
```

	species	count
0	Adelie	152
1	Chinstrap	68
2	Gentoo	124

```
1 fig, ax = plt.subplots(figsize=(8, 6))  
2 ax.bar(  
3     x="species",  
4     height="count",  
5     data=bp1_data,  
6     color="steelblue"  
7 )  
8 ax.set(  
9     xlabel="Species",  
10    ylabel="Count",  
11    title="Penguin Species Counts")  
12 plt.show()
```

Bar Plots



Bar plot of penguin species counts for the penguins dataset. (`matplotlib`)

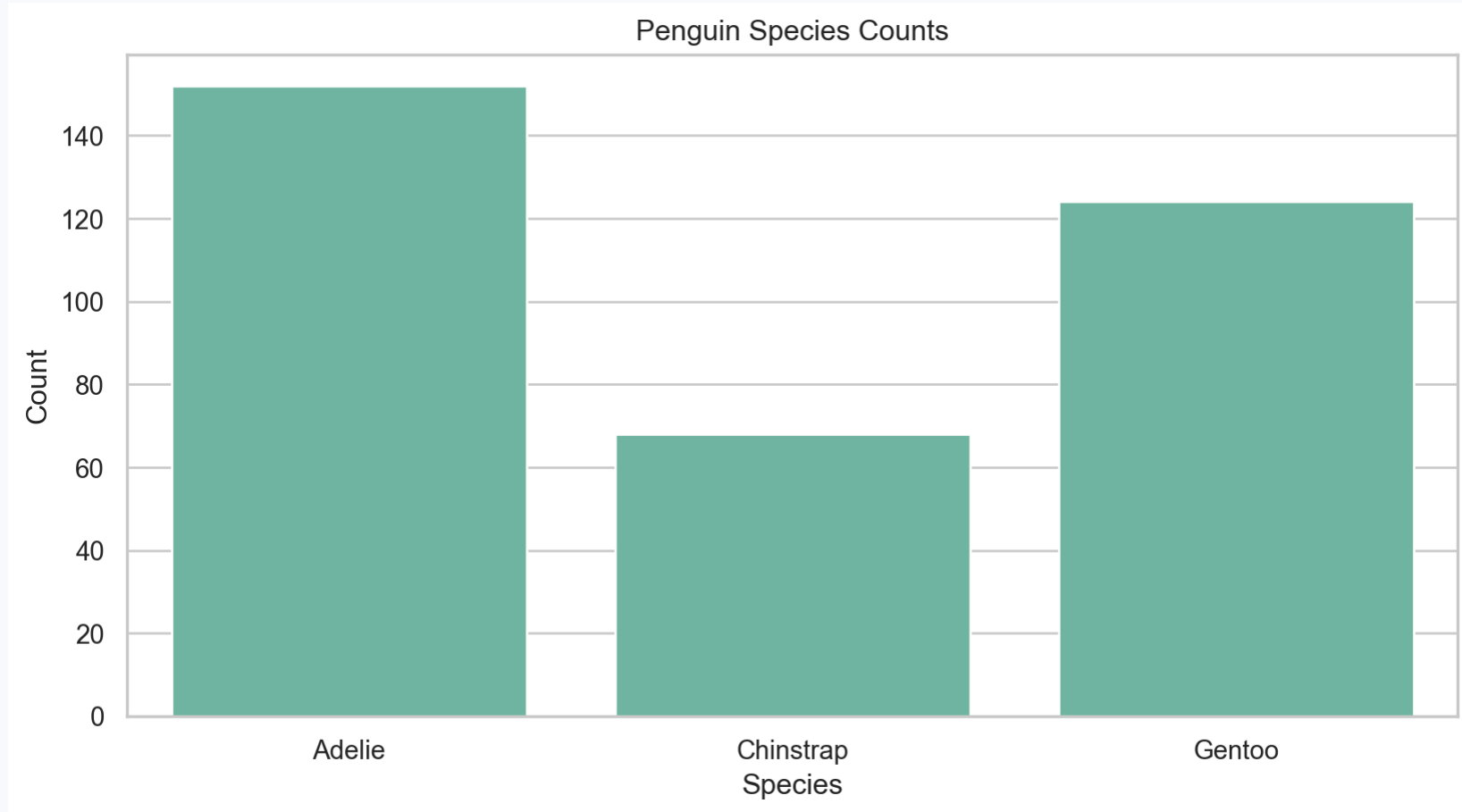
Count Plots

seaborn Count Plots

- Although `seaborn` has an equivalent `barplot` function, a more useful plot type in this case is a `countplot`.
- This automatically computes the counts for each category and creates a bar plot without needing to pre-aggregate the data.

```
1 ax = sns.countplot(  
2     x="species",  
3     data=penguins  
4 )  
5 ax.set(  
6     xlabel="Species",  
7     ylabel="Count",  
8     title="Penguin Species Counts")  
9 plt.show()
```

Count Plots



Bar plot of penguin species counts for the penguins dataset. ([seaborn](#))

Subgrouping and Variable Formatting

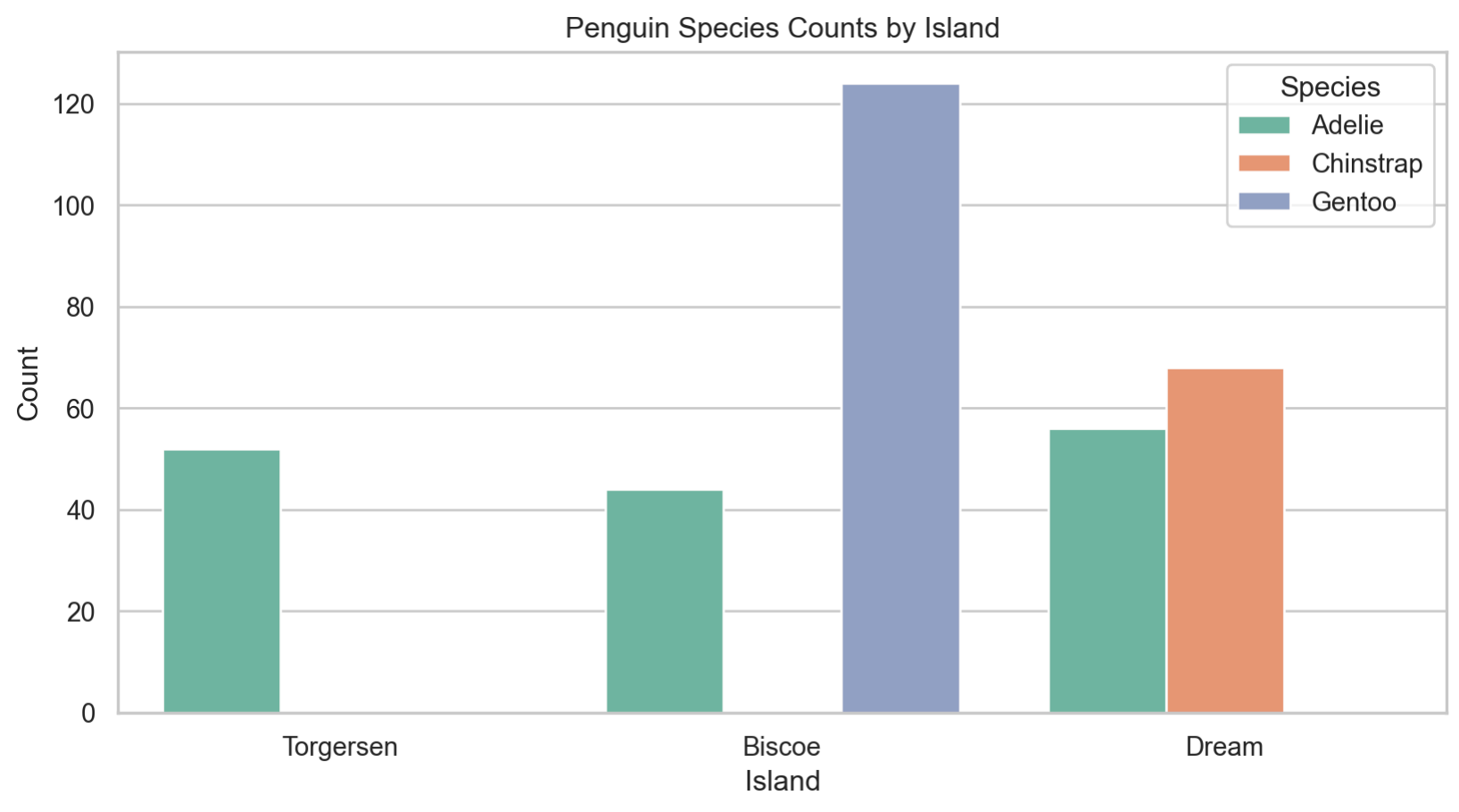
Subgrouping

- To examine relationships between multiple categorical variables, we can split each bar into subgroups based on a second categorical variable (e.g., species and island).

Subgrouped Bar Plot with Variable Formatting

```
1 ax = sns.countplot(  
2     x="island",  
3     hue="species",  
4     data=penguins  
5 )  
6 ax.set(  
7     xlabel="Island",  
8     ylabel="Count",  
9     title="Penguin Species Counts by Island"  
10 )  
11 ax.legend(title="Species")  
12 plt.show()
```

Subgrouping and Variable Formatting



Bar plot of penguin species counts for the penguins dataset. (`seaborn`)

Box Plots

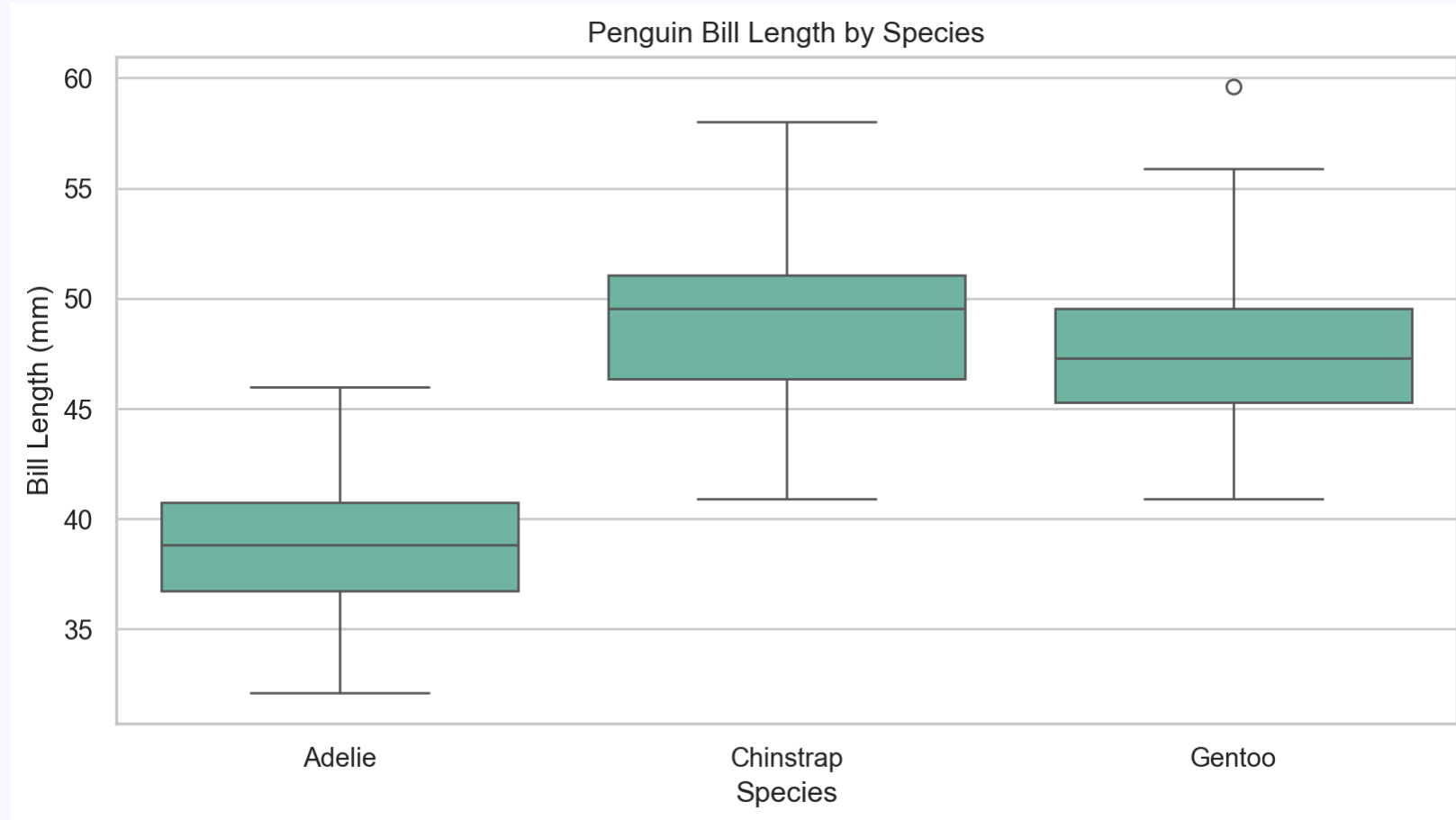
What are box plots?

- A **box plot** (or box-and-whisker plot) is a standardized way of displaying the distribution of data based on a five-number summary: minimum, first quartile (Q1), median, third quartile (Q3), and maximum.
- It can also show **outliers** in the data.
- In `seaborn`, we can produce a box plot using the `boxplot()` function.

`seaborn` Box Plots

```
1 ax = sns.boxplot(  
2     x="species", y="bill_length_mm",  
3     data=penguins  
4 )  
5 ax.set(  
6     xlabel="Species",  
7     ylabel="Bill Length (mm)",  
8     title="Penguin Bill Length by Species"  
9 )  
10 plt.show()
```

Box Plots



Box plot of bill length by species for the penguins dataset. ([seaborn](#))

Histograms

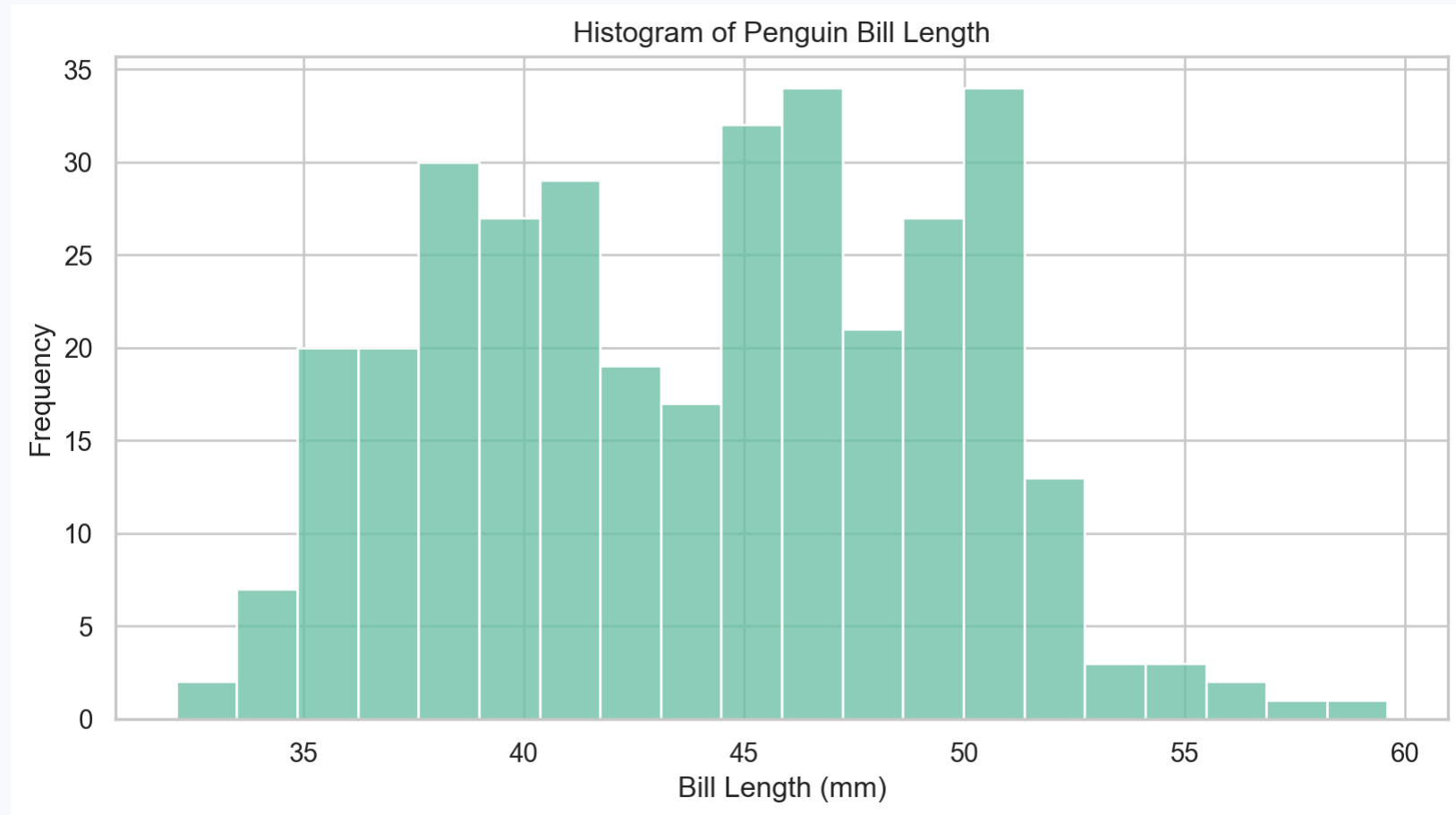
What are histograms?

- A **histogram** is a graphical representation of the distribution of a dataset, where the data is divided into bins and the frequency of data points in each bin is represented by the height of the bar.
- In `seaborn`, we can produce a histogram using the `histplot()` function.

`seaborn` Histograms

```
1 ax = sns.histplot(  
2     x="bill_length_mm",  
3     data=penguins,  
4     bins=20          # Set number of bins  
5 )  
6 ax.set(xlabel="Bill Length (mm)", ylabel="Frequency", title="Histogram of Penguin Bill Length")  
7 plt.show()
```

Histograms



Histogram of bill length for the penguins dataset. (`seaborn`)

Histograms with Variable Formatting and Kernel Density Estimates

Let's start exploring what we can do!

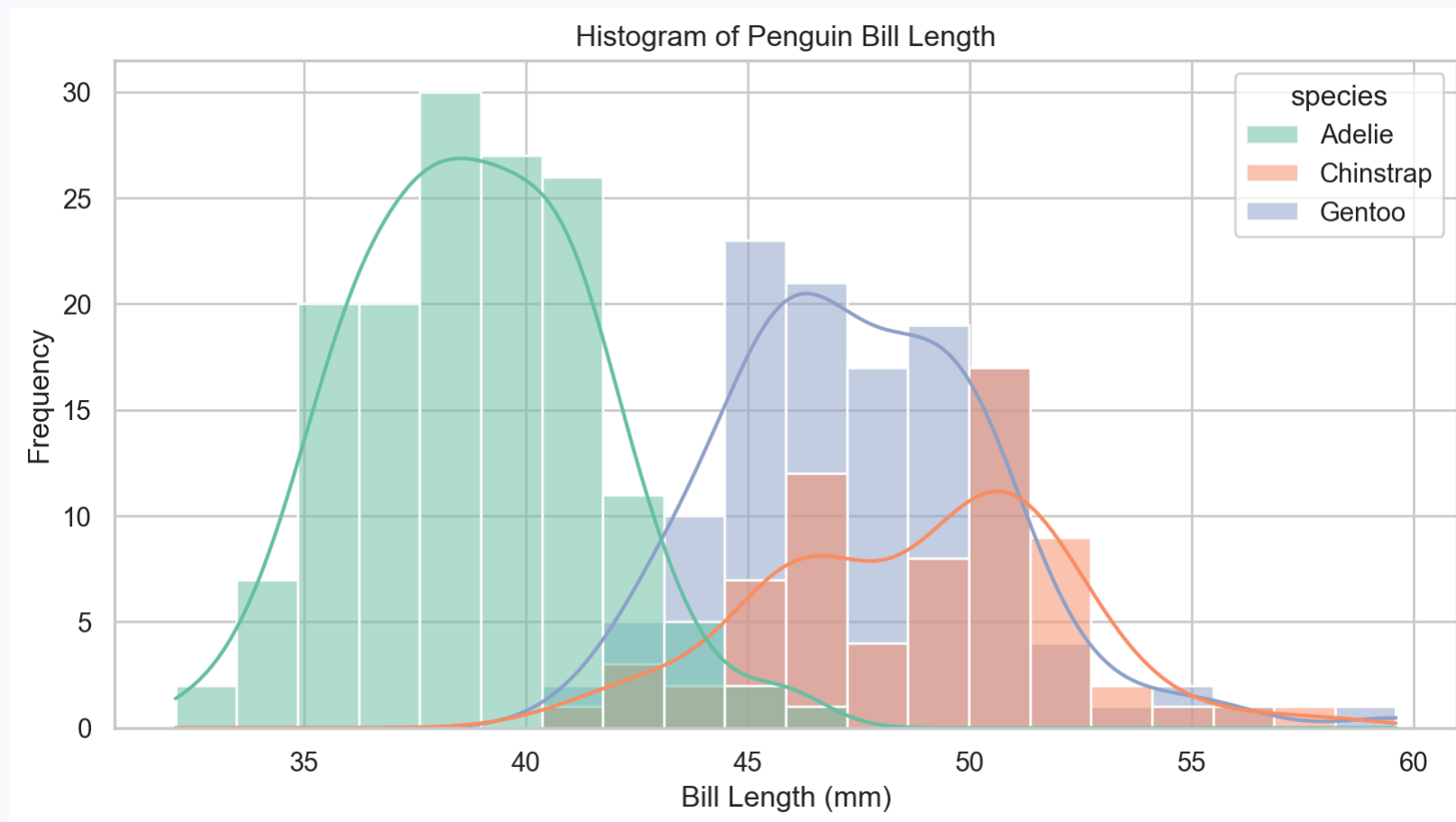
- We know that the three species of penguins have different bill length distributions.
 - Can we visualize this in a single plot?
- We can produce **kernel density estimates (KDEs)** to visualize the distribution of bill length for each species in a single plot.

seaborn Histograms with KDEs

- Histogram shape depends on binning choices, and KDE smoothness depends on bandwidth, so use these as exploratory summaries rather than definitive evidence.

```
1 ax = sns.histplot(  
2     x="bill_length_mm",  
3     data=penguins,  
4     bins=20,           # Set number of bins  
5     kde=True,          # Show kernel density estimate  
6     hue="species",     # Color by species  
7     multiple="layer"   # Overlay histograms  
8 )  
9 ax.set(xlabel="Bill Length (mm)", ylabel="Frequency", title="Histogram of Penguin Bill Length")  
10 plt.show()
```

Histograms with Variable Formatting and Kernel Density Estimates



Histogram of bill length for the penguins dataset. ([seaborn](#))

Violin Plots

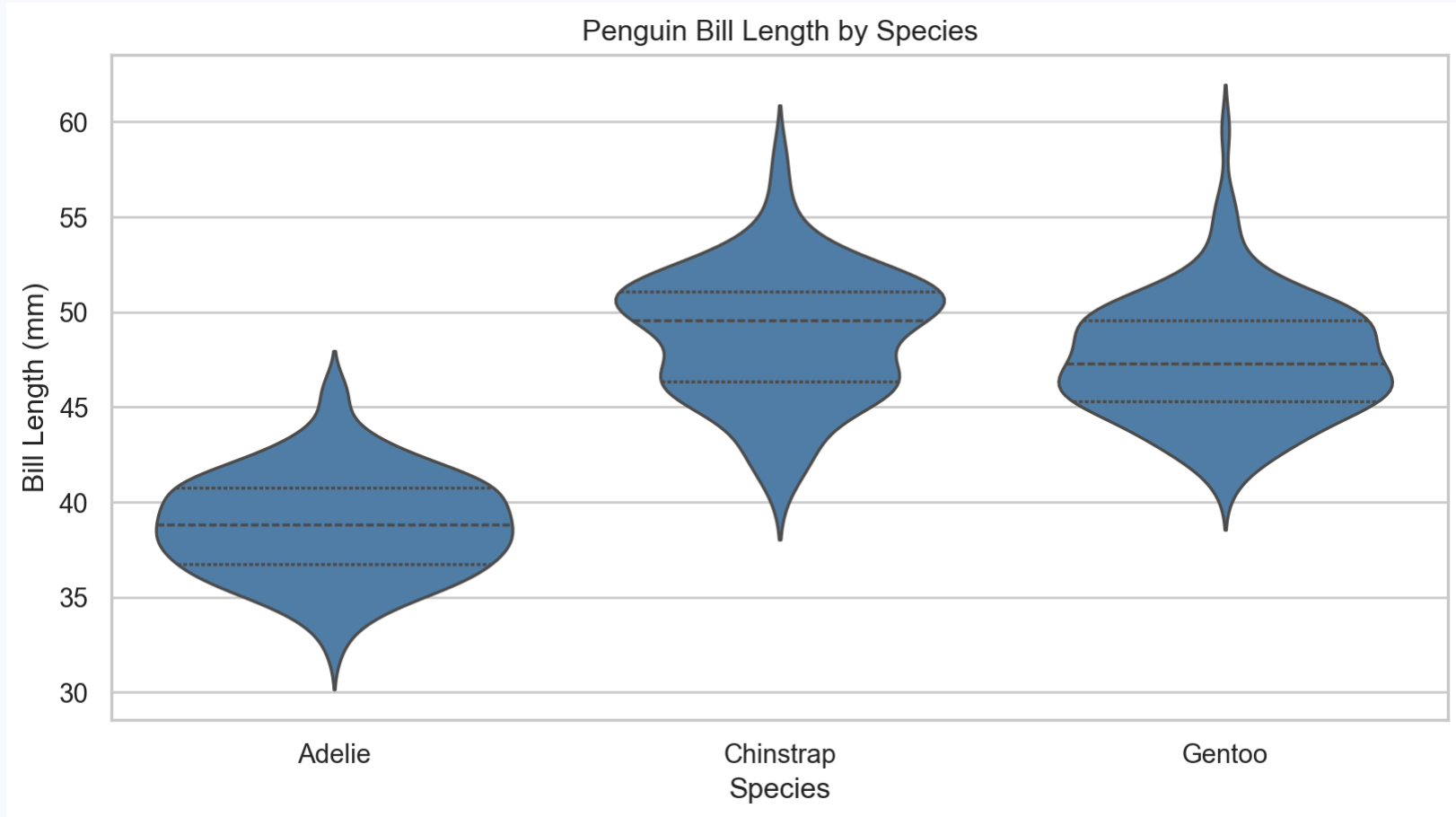
Fancy box plots?

- A **violin plot** is a method of plotting numeric data and can be understood as a combination of a box plot and a kernel density plot (a smoothed version of a histogram).
- It provides a visual summary of the distribution of the data, including its central tendency, variability, and shape.
- In `seaborn`, we can produce a violin plot using the `violinplot()` function.

Violin Plots in `seaborn`

```
1 ax = sns.violinplot(  
2     x="species", y="bill_length_mm",  
3     data=penguins,  
4     inner="quartile",           # Show quartiles inside the violin  
5     color="steelblue"         # Set violin color  
6 )  
7 ax.set(  
8     xlabel="Species",  
9     ylabel="Bill Length (mm)",  
10    title="Penguin Bill Length by Species"  
11 )  
12 plt.show()
```

Violin Plots



Violin plot of bill length by species for the penguins dataset. ([seaborn](#))

Advanced Visualizations

Advanced Visualizations

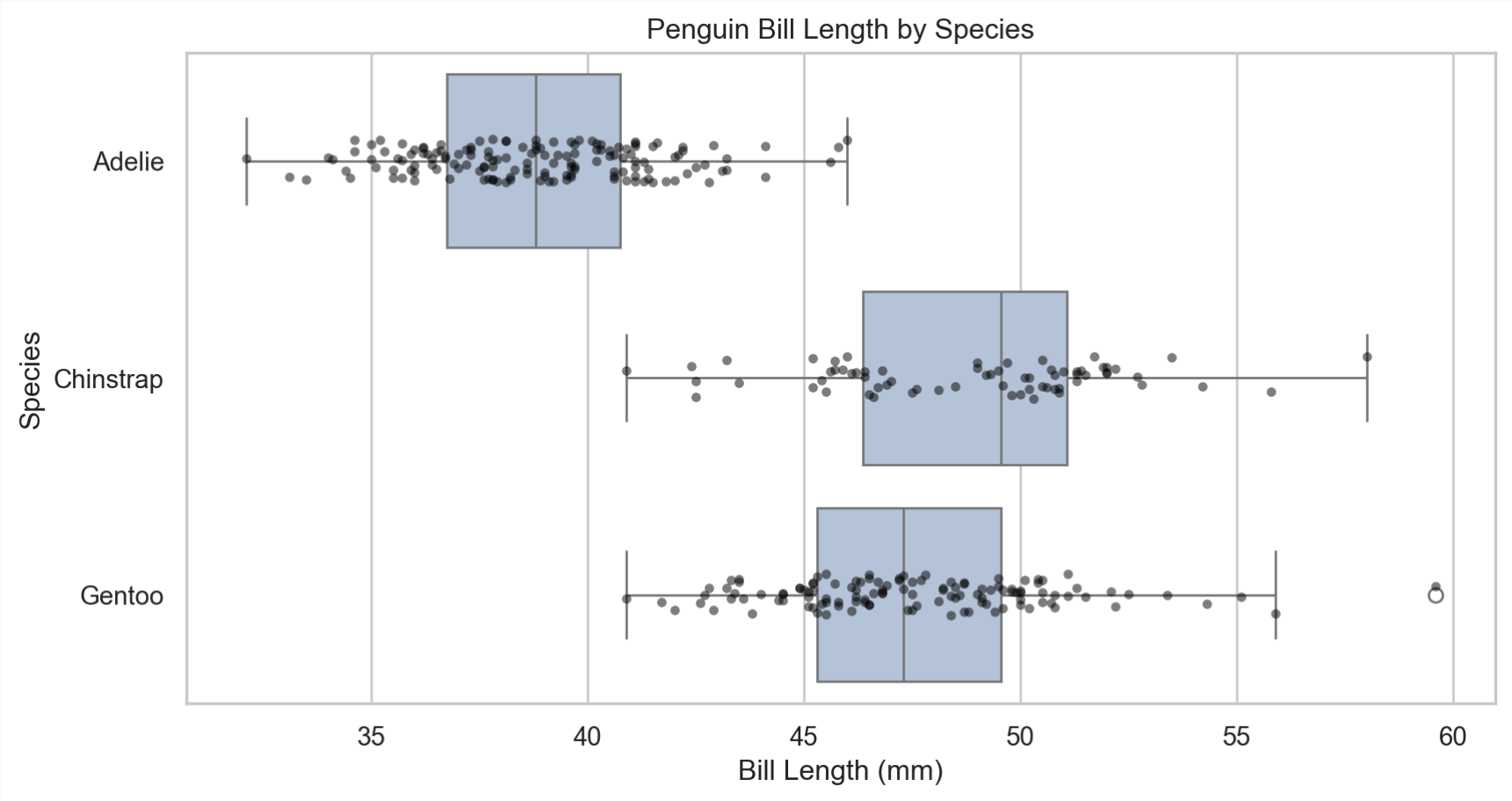
Be creative!

- Remember, the goal of visualization is to communicate in a novel, aesthetic, and informative way.
- Don't be afraid to experiment with different visualizations.
- Tweak and change things until they are clear!

Perhaps this boxplot is more informative?

```
1 ax = sns.boxplot(  
2     x="bill_length_mm", y="species",  
3     data=penguins,  
4     color="lightsteelblue"  
5 )  
6 sns.stripplot(  
7     x="bill_length_mm", y="species",  
8     data=penguins,  
9     color="black",           # Use a single point color for readability  
10    alpha=0.5,              # Set point transparency  
11    size=4,                 # Set point size  
12    ax=ax  
13 )  
14 ax.set(  
15     xlabel="Bill Length (mm)",  
16     ylabel="Species",  
17     title="Penguin Bill Length by Species"  
18 )  
19 plt.show()
```

Advanced Visualizations



Box plot of bill length by species for the penguins dataset. ([seaborn](#))

Continuous Variable Formatting

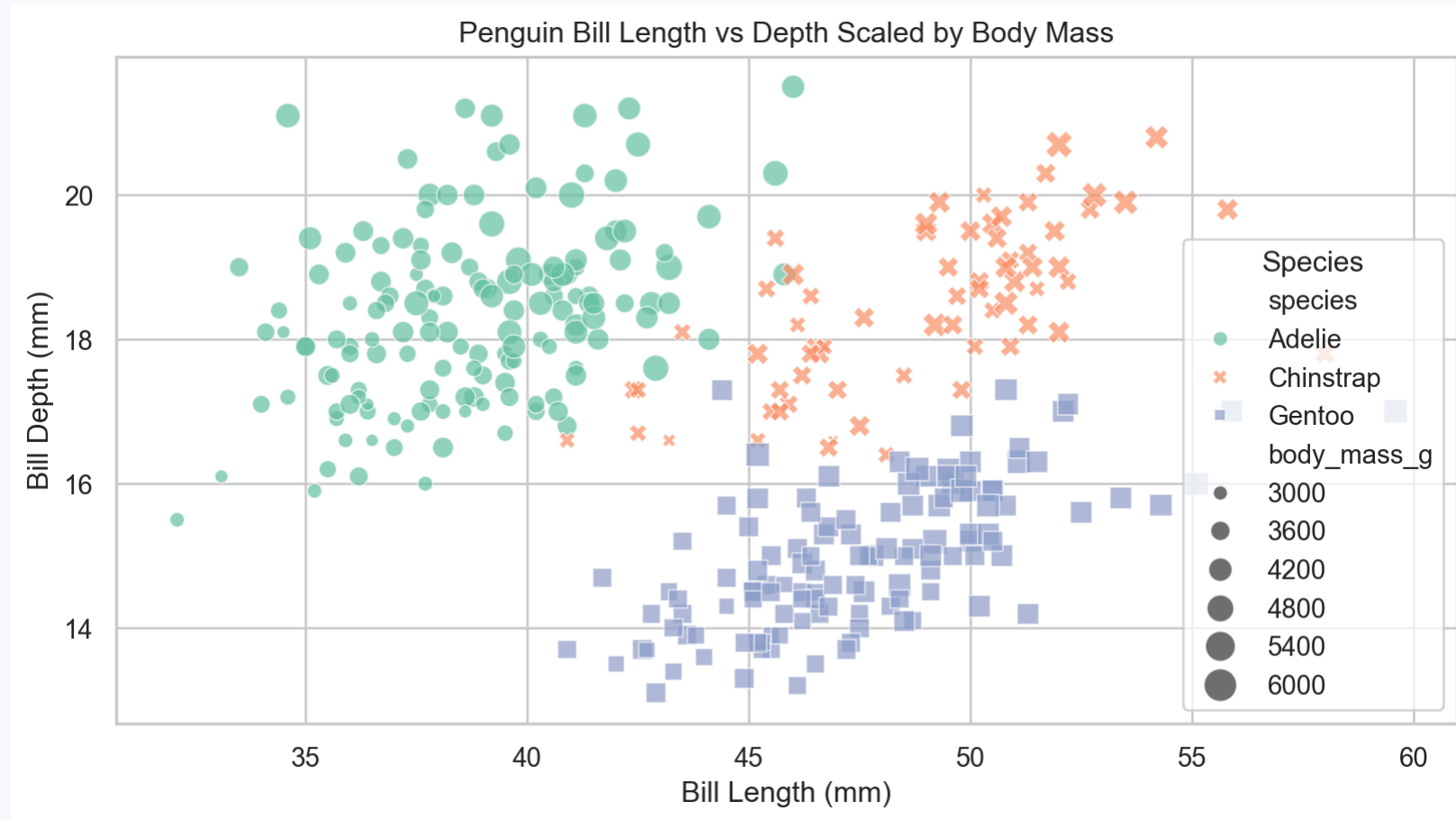
We can also format using continuous variables!

- For our scatter plot suppose we wish to also incorporate the influence of body mass on bill length and depth.
- We can use the `size` and `sizes` parameters to control the size of points based on body mass.

How about this scatter plot?

```
1 ax = sns.scatterplot(  
2     x="bill_length_mm", y="bill_depth_mm",  
3     data=penguins,  
4     hue="species",           # Color by species  
5     style="species",         # Shape by species  
6     size="body_mass_g",      # Scale by body mass  
7     sizes=(20, 200),        # Set size range  
8     alpha=0.7               # Set point transparency  
9 )  
10 ax.set(xlabel="Bill Length (mm)", ylabel="Bill Depth (mm)", title="Penguin Bill Length vs Depth Scaled by Body Mass")  
11 ax.legend(title="Species")  
12 plt.show()
```


Continuous Variable Formatting



Scatter plot of bill length vs bill depth scaled by body mass for the penguins dataset. ([seaborn](#))

Multiplotting

Multiplotting

What is multiplotting?

- Particularly for reports we often are interested in showing multiple related visualizations together.
- In `matplotlib` we can use the `subplots()` function to create multiple plots in a single figure.
- In `seaborn` we can use the `FacetGrid()` function to create a grid of plots based on the values of one or more categorical variables.
 - This is where the improved efficiency of `seaborn` becomes apparent as we can create complex multiplots with very little code.

Multiplots with `matplotlib`

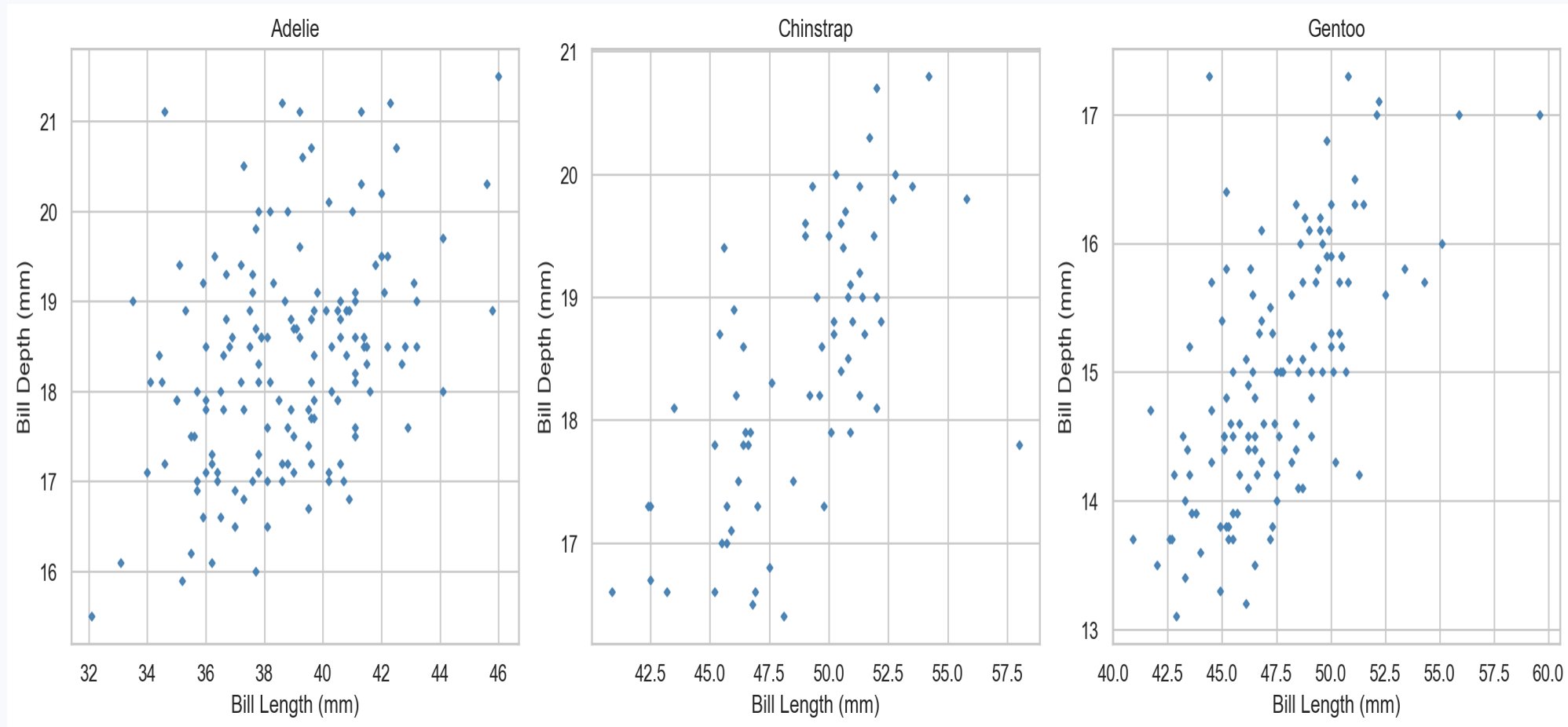
It's still important to learn the syntax

- We first create a figure with multiple axes using `plt.subplots()`.
- We then loop through the axes and create a scatter plot for each species in the penguins dataset.
- Finally, we adjust the layout and display the plot.

Starting to get confusing! 🤪

```
1 fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(15, 5))      # Create figure and axes objects
2 species = penguins["species"].unique()                          # Get unique species
3
4 for ax, sp in zip(axes, species):                                # Loop through axes and species
5     subset = penguins[penguins["species"] == sp]                # Subset data for each species
6     ax.scatter(                                                  # Create scatter plot
7         subset["bill_length_mm"], subset["bill_depth_mm"],
8         color="steelblue",
9         alpha=0.95,
10        s=7,
11        marker="D"
12    )
13    ax.set_title(sp)
14    ax.set_xlabel("Bill Length (mm)")
15    ax.set_ylabel("Bill Depth (mm)")
16 plt.tight_layout()                                              # Adjust layout
17 plt.show()
```

Multiplots with `matplotlib`



Multiplot of bill length vs bill depth for each species in the penguins dataset. (`matplotlib`)

Multiplots with `seaborn`

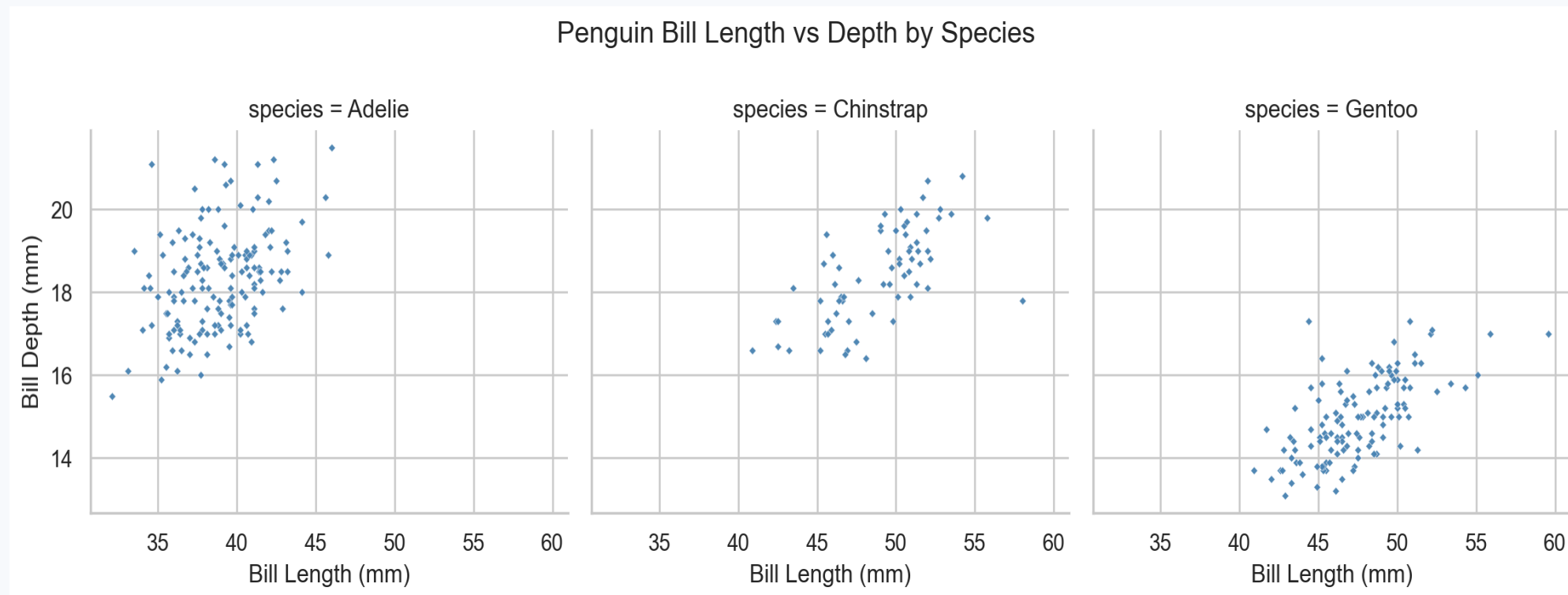
So much easier! 😎

- We can create the same multiplot using `seaborn`'s `FacetGrid()` function with much less code.
- This function allows us to create a grid of plots based on the values of one or more categorical variables, making it easy to compare relationships across different groups.
- The `map()` method is used to apply a plotting function (e.g., `sns.scatterplot`) to each subset of the data defined by the grid.

Facets with `seaborn`

```
1 g = sns.FacetGrid(
2     penguins,
3     col="species",
4     height=4,
5     aspect=1) # Create facet grid
6 g.map(
7     sns.scatterplot,
8     "bill_length_mm", "bill_depth_mm",
9     color="steelblue",
10    alpha=0.95,
11    s=7,
12    marker="D") # Map scatter plot to each facet
13 g.set_axis_labels(
14     "Bill Length (mm)",
15     "Bill Depth (mm)"
16 ) # Set axis labels
17 g.fig.subplots_adjust(top=0.8) # Adjust subplot spacing
```

Multiplots with `seaborn`



Multiplot of bill length vs bill depth for each species in the penguins dataset. (`seaborn`)

Conclusion

✓ What we covered

- Standard visualizations and when to use them.
- Basic visualization production in `matplotlib` and `seaborn`.
- Style and formatting features including themes, variable formatting, and facets.
- The importance of creativity and experimentation in data visualization.
- Facets.

What's next?

- **Exploratory Data Analysis (EDA).**
- Descriptive statistics.
- Complex `seaborn` visualizations.
- Interactive visualizations with `altair`.

References